DevOps Lab Record

Experiment 1: Simple User Registration Form for an Event

Aim: To design and develop a simple user registration form for an event using HTML, CSS and JavaScript with client-side validation.

Software required: VS Code (or any text editor), Google Chrome / Firefox.

Theory: HTML defines the structure of a web page using elements such as `<form>`, `<input>`, `<select>` and `<button>`. CSS controls presentation (layout, colours, fonts). JavaScript adds behaviour; here it validates the input before submission and shows a confirmation. HTML5 attributes such as `required`, `type="email"`, `pattern` and `min` give built-in validation.

Procedure:

1. Create a folder named `event-registration`.
2. Inside it create three files: `index.html`, `style.css` and `script.js`.
3. Type the code given below into each file and save.
4. Open `index.html` in a browser (double-click, or use the VS Code Live Server extension).
5. Test the form with valid and invalid inputs and note the results.

Program:

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TechFest 2026 - Registration</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>TechFest 2026</h1>
    <p class="sub">Event Registration Form</p>

    <form id="regForm" novalidate>
      <label for="name">Full Name</label>
      <input type="text" id="name" placeholder="Enter your full name" required>

      <label for="email">Email</label>
      <input type="email" id="email" placeholder="name@example.com" required>

      <label for="phone">Mobile Number</label>
      <input type="tel" id="phone" placeholder="10-digit mobile number" required>

      <label for="college">College</label>
      <input type="text" id="college" placeholder="College name" required>

      <label for="event">Select Event</label>
      <select id="event" required>
        <option value="">-- Choose an event --</option>
        <option>Hackathon</option>
        <option>Paper Presentation</option>
```

```
        <option>Coding Contest</option>
        <option>Robotics Workshop</option>
      </select>

      <label>Gender</label>
      <div class="radio">
        <input type="radio" name="gender" value="Male" id="m"><label
for="m">Male</label>
        <input type="radio" name="gender" value="Female" id="f"><label
for="f">Female</label>
        <input type="radio" name="gender" value="Other" id="o"><label
for="o">Other</label>
      </div>

      <label for="password">Password</label>
      <input type="password" id="password" placeholder="Min. 6 characters"
required>

      <div class="check">
        <input type="checkbox" id="terms">
        <label for="terms">I agree to the terms and conditions</label>
      </div>

      <p id="error" class="error"></p>
      <button type="submit">Register</button>
      <button type="reset" class="reset">Clear</button>
    </form>

    <div id="success" class="success"></div>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body {
  font-family: Arial, sans-serif;
  background: linear-gradient(135deg, #4b6cb7, #182848);
  min-height: 100vh;
  display: flex; justify-content: center; align-items: center;
}
.container {
  background: #fff; width: 400px; padding: 30px;
  border-radius: 10px; box-shadow: 0 8px 20px rgba(0,0,0,0.3);
}
h1 { text-align: center; color: #182848; }
.sub { text-align: center; color: #666; margin-bottom: 20px; }
label { display: block; margin-top: 12px; font-weight: bold; font-size: 14px; }
input[type=text], input[type=email], input[type=tel],
input[type=password], select {
  width: 100%; padding: 9px; margin-top: 5px;
  border: 1px solid #ccc; border-radius: 5px;
}
.radio label, .check label { display: inline; font-weight: normal; margin-right:
10px; }
.radio, .check { margin-top: 8px; }
button {
  width: 48%; padding: 10px; margin-top: 18px; border: none;
```

```
  border-radius: 5px; background: #4b6cb7; color: #fff; cursor: pointer;
}
button.reset { background: #999; }
button:hover { opacity: 0.85; }
.error { color: red; font-size: 13px; margin-top: 10px; }
.success { color: green; margin-top: 15px; font-weight: bold; }
```

script.js

```
document.getElementById("regForm").addEventListener("submit", function (e) {
  e.preventDefault();

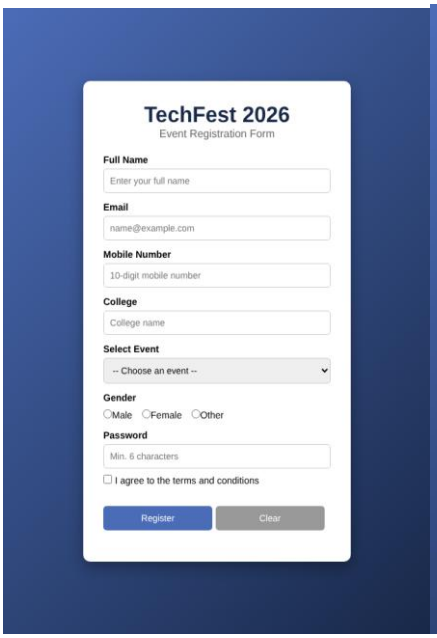
  const name     = document.getElementById("name").value.trim();
  const email    = document.getElementById("email").value.trim();
  const phone    = document.getElementById("phone").value.trim();
  const college  = document.getElementById("college").value.trim();
  const event    = document.getElementById("event").value;
  const gender   = document.querySelector('input[name="gender"]:checked');
  const password = document.getElementById("password").value;
  const terms    = document.getElementById("terms").checked;
  const error    = document.getElementById("error");
  const success  = document.getElementById("success");

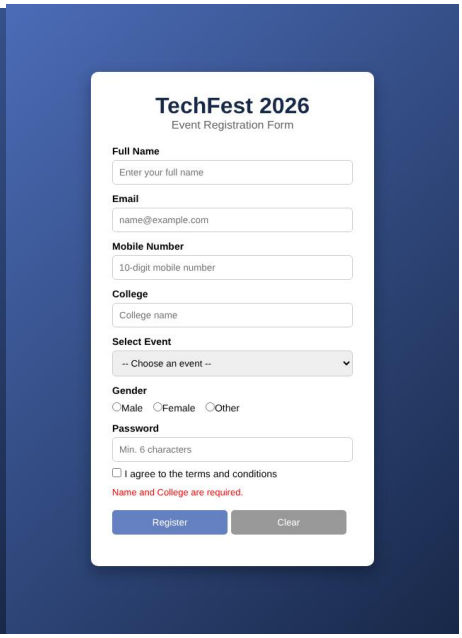
  error.textContent = "";
  success.textContent = "";

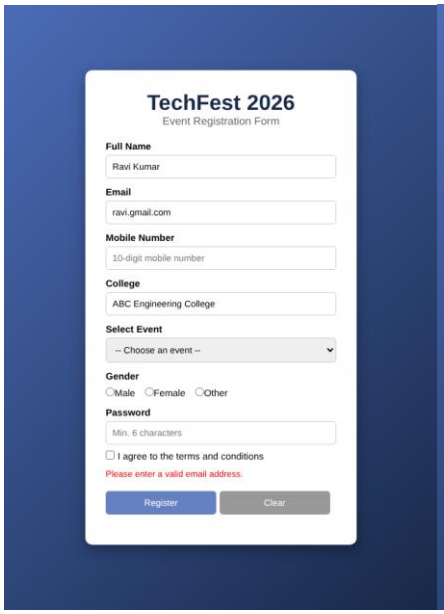
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  const phonePattern = /^[6-9][0-9]{9}$/;

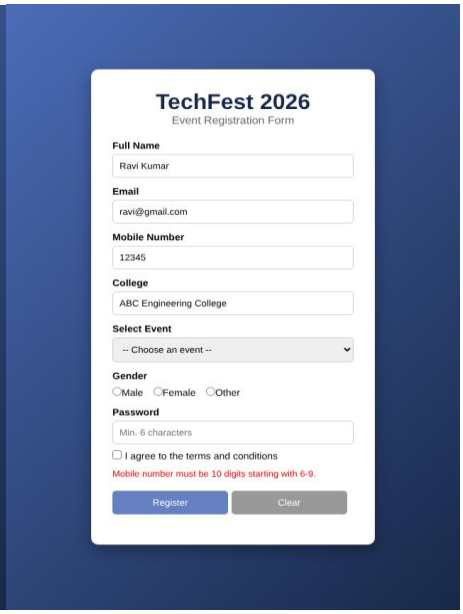
  if (name === "" || college === "") {
    error.textContent = "Name and College are required.";
  } else if (!emailPattern.test(email)) {
    error.textContent = "Please enter a valid email address.";
  } else if (!phonePattern.test(phone)) {
    error.textContent = "Mobile number must be 10 digits starting with 6-9.";
  } else if (event === "") {
    error.textContent = "Please select an event.";
  } else if (!gender) {
    error.textContent = "Please select your gender.";
  } else if (password.length < 6) {
    error.textContent = "Password must be at least 6 characters.";
  } else if (!terms) {
    error.textContent = "You must accept the terms and conditions.";
  } else {
    success.textContent = "Thank you " + name + "! You are registered for " +
event + ".";
    document.getElementById("regForm").reset();
  }
});
```

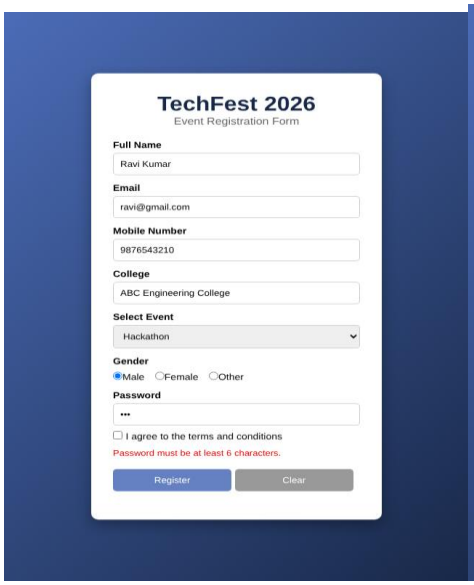
Output:

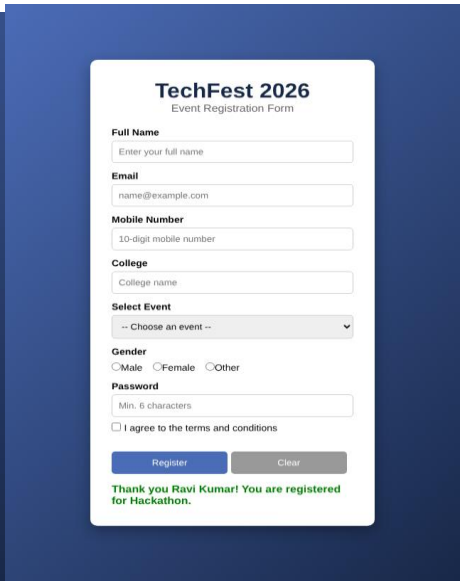

TechFest 2026
Event Registration Form
Full Name
Enter your full name
Email
name@example.com
Mobile Number
10-digit mobile number
College
College name
Select Event
-- Choose an event --
Gender
Male Female Other
Password
Min. 6 characters
I agree to the terms and conditions
Register
Clear


TechFest 2026
Event Registration Form
Full Name
Enter your full name
Email
name@example.com
Mobile Number
10-digit mobile number
College
College name
Select Event
-- Choose an event --
Gender
Male Female Other
Password
Min. 6 characters
I agree to the terms and conditions
Name and College are required.
Register
Clear


TechFest 2026
Event Registration Form
Full Name
Ravi Kumar
Email
ravi.gmail.com
Mobile Number
10-digit mobile number
College
ABC Engineering College
Select Event
-- Choose an event --
Gender
Male Female Other
Password
Min. 6 characters
I agree to the terms and conditions
Please enter a valid email address.
Register
Clear


TechFest 2026
Event Registration Form
Full Name
Ravi Kumar
Email
ravi@gmail.com
Mobile Number
12345
College
ABC Engineering College
Select Event
-- Choose an event --
Gender
Male Female Other
Password
Min. 6 characters
I agree to the terms and conditions
Mobile number must be 10 digits starting with 6-9.
Register
Clear


TechFest 2026
Event Registration Form
Full Name
Ravi Kumar
Email
ravi@gmail.com
Mobile Number
9876543210
College
ABC Engineering College
Select Event
Hackathon
Gender
Male Female Other
Password
I agree to the terms and conditions
Password must be at least 6 characters.
Register
Clear


TechFest 2026
Event Registration Form
Full Name
Enter your full name
Email
name@example.com
Mobile Number
10-digit mobile number
College
College name
Select Event
-- Choose an event --
Gender
Male Female Other
Password
Min. 6 characters
I agree to the terms and conditions
Register
Clear
Thank you Ravi Kumar! You are registered for Hackathon.

Result: A user registration form for an event was designed using HTML, CSS and JavaScript, and its client-side validation was verified successfully with valid and invalid inputs.

Experiment 2: Exploring Git and GitHub Commands

Aim: To install Git and explore the basic Git and GitHub commands used for version control.

Software required: Git 2.4x, a GitHub account, a terminal (Git Bash on Windows, Terminal on Linux/macOS).

Theory: Git is a distributed version control system that records changes to files as snapshots called commits. A file moves through three areas: the working directory (where you edit), the staging area (`git add`) and the local repository (`git commit`). GitHub is a cloud hosting service for Git repositories; `git push` and `git pull` move commits between the local repository and the remote repository on GitHub.

Procedure:

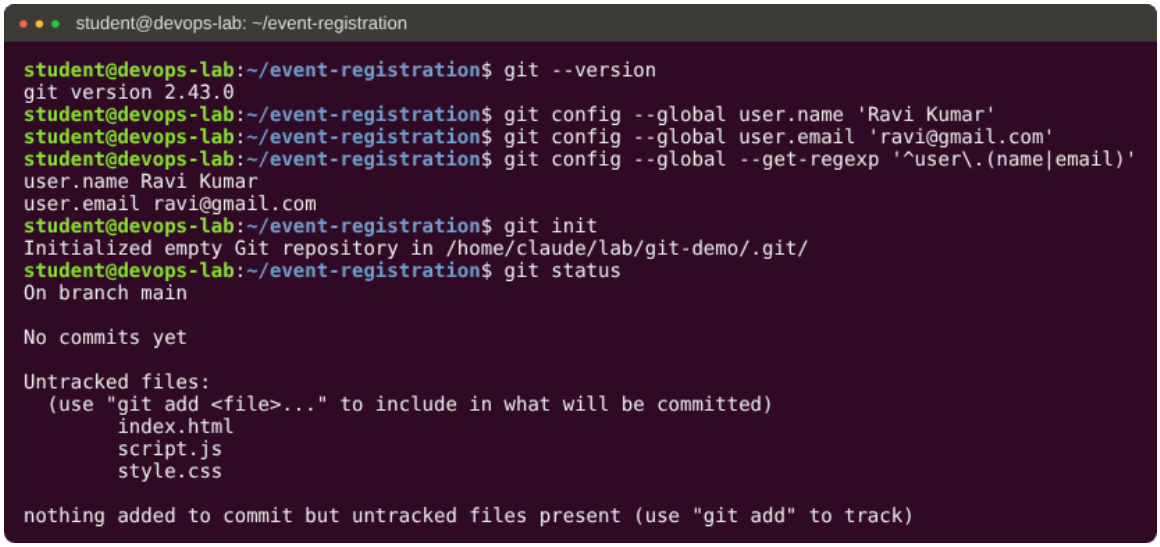
1. Download Git from git-scm.com and install it with default options. Verify with `git --version`.
2. Configure your identity with `git config`.
3. Create a GitHub account at github.com.
4. Execute each command in the table below and record its output.

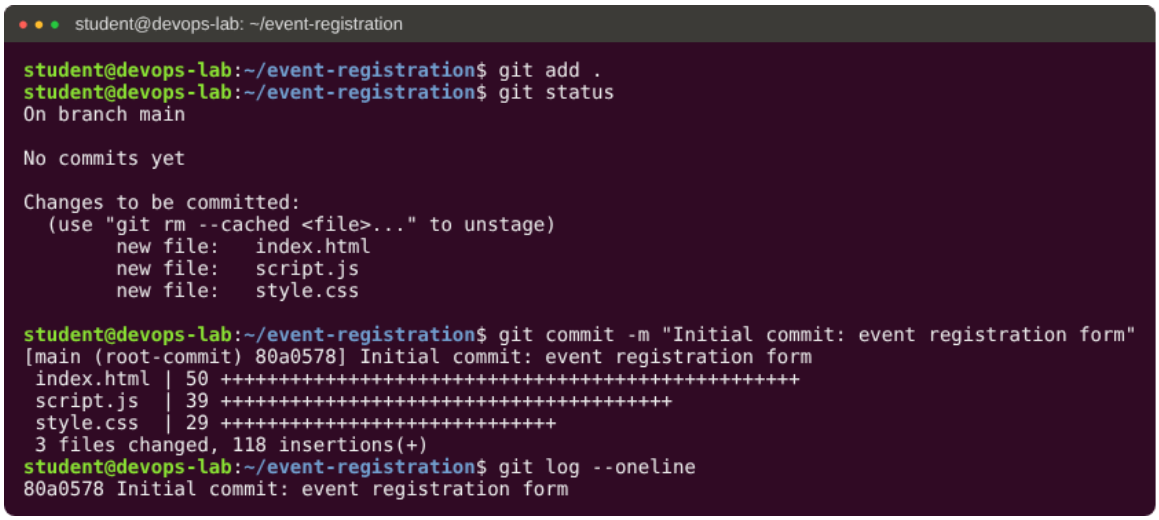
Commands explored:

Command	Purpose
`git --version`	Show installed Git version
`git config --global user.name "Name"`	Set author name for commits
`git config --global user.email "email"`	Set author email for commits
`git config --list`	List all configuration settings
`git init`	Create a new local repository
`git status`	Show state of working directory and staging area
`git add <file>` / `git add .`	Stage file(s) for the next commit
`git commit -m "msg"`	Save staged changes as a commit
`git log` / `git log --oneline`	Show commit history
`git diff`	Show unstaged changes line by line
`git branch <name>` / `git branch`	Create / list branches
`git checkout <branch>` or `git switch <branch>`	Switch to another branch
`git merge <branch>`	Merge a branch into the current branch

Command	Purpose
`git remote add origin <url>` / `git remote -v`	Link / show remote repository
`git push -u origin main`	Upload local commits to GitHub
`git pull origin main`	Fetch and merge changes from GitHub
`git clone <url>`	Copy a remote repository to the local machine
`git tag v1.0`	Mark a release point
`git show`	Show details of a commit
`git rm <file>` / `git mv <old> <new>`	Remove / rename a tracked file
`git reset <file>`	Unstage a file
`git stash` / `git stash pop`	Save and restore uncommitted work temporarily

Output:





```
student@devops-lab: ~/event-registration

student@devops-lab:~/event-registration$ git diff
diff --git a/style.css b/style.css
index c3694f4..4dd9b15 100644
--- a/style.css
+++ b/style.css
@@ -21,7 +21,7 @@ input[type=password], select {
 .radio, .check { margin-top: 8px; }
 button {
   width: 48%; padding: 10px; margin-top: 18px; border: none;
-  border-radius: 5px; background: #4b6cb7; color: #fff; cursor: pointer;
+  border-radius: 5px; background: #2e7d32; color: #fff; cursor: pointer;
 }
 button.reset { background: #999; }
 button:hover { opacity: 0.85; }
student@devops-lab:~/event-registration$ git commit -am "Update button style"
[main dc70cf8] Update button style
 style.css | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
student@devops-lab:~/event-registration$ git branch feature-validation
student@devops-lab:~/event-registration$ git branch
  feature-validation
* main
student@devops-lab:~/event-registration$ git remote add origin https://github.com/ravikumar/event-registration.git
student@devops-lab:~/event-registration$ git remote -v
origin  https://github.com/ravikumar/event-registration.git (fetch)
origin  https://github.com/ravikumar/event-registration.git (push)
student@devops-lab:~/event-registration$ git tag v1.0
student@devops-lab:~/event-registration$ git tag
v1.0
```

Result: Git was installed and configured, and the basic Git and GitHub commands for creating a repository, staging, committing, viewing history, branching and linking a remote were executed successfully.

Experiment 3: Source Code Management on GitHub

Aim: To practice source code management on GitHub by pushing the registration form code from Experiment 1, working on feature branches, merging, resolving a merge conflict and raising a pull request.

Software required: Git, GitHub account, VS Code, source code of Experiment 1.

Theory: Source Code Management (SCM) tracks every change to code, lets several developers work in parallel on branches and merges their work safely. GitHub adds collaboration features on top of Git: remote repositories, pull requests (PRs) for code review, issues and forks. A merge conflict occurs when two branches change the same line; Git marks the conflict with `<<<<<<<`, `=======` and `>>>>>>>` and the developer resolves it manually.

Procedure:

1. Log in to GitHub, click **New repository**, name it `event-registration`, keep it Public and click **Create repository** (do not add a README).
2. In the `event-registration` folder from Experiment 1, initialize Git and make the first commit:

```
git init
git add .
git commit -m "Initial commit: event registration form"
```

3. Link the GitHub repository and push:

```
git remote add origin https://github.com/<username>/event-registration.git
git branch -M main
git push -u origin main
```

When asked for a password, paste a GitHub Personal Access Token (Settings → Developer settings → Personal access tokens).

4. Modify `style.css` (change the button colour), then commit and push:

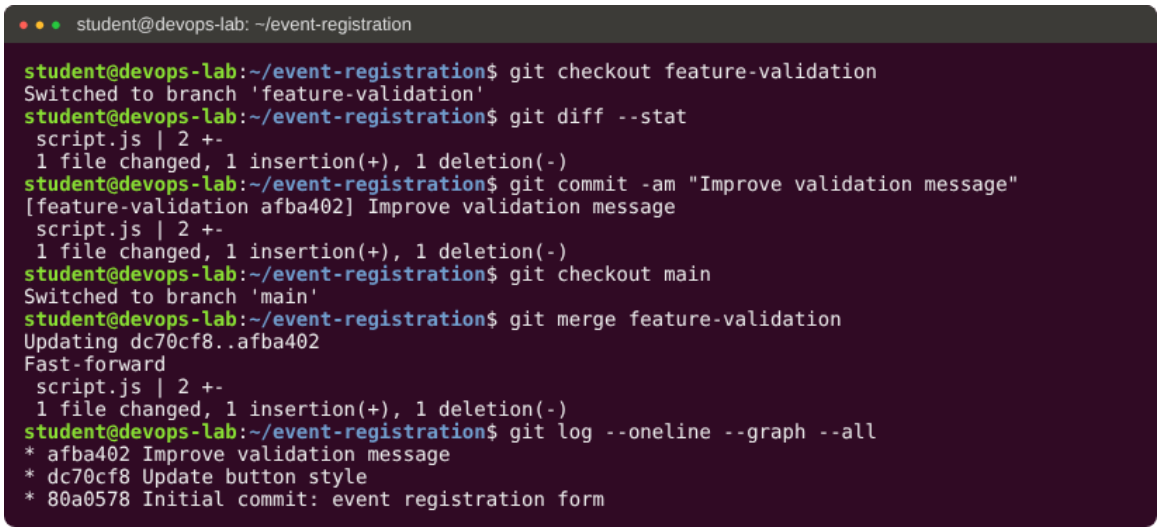
```
git commit -am "Update button style"
git push
```

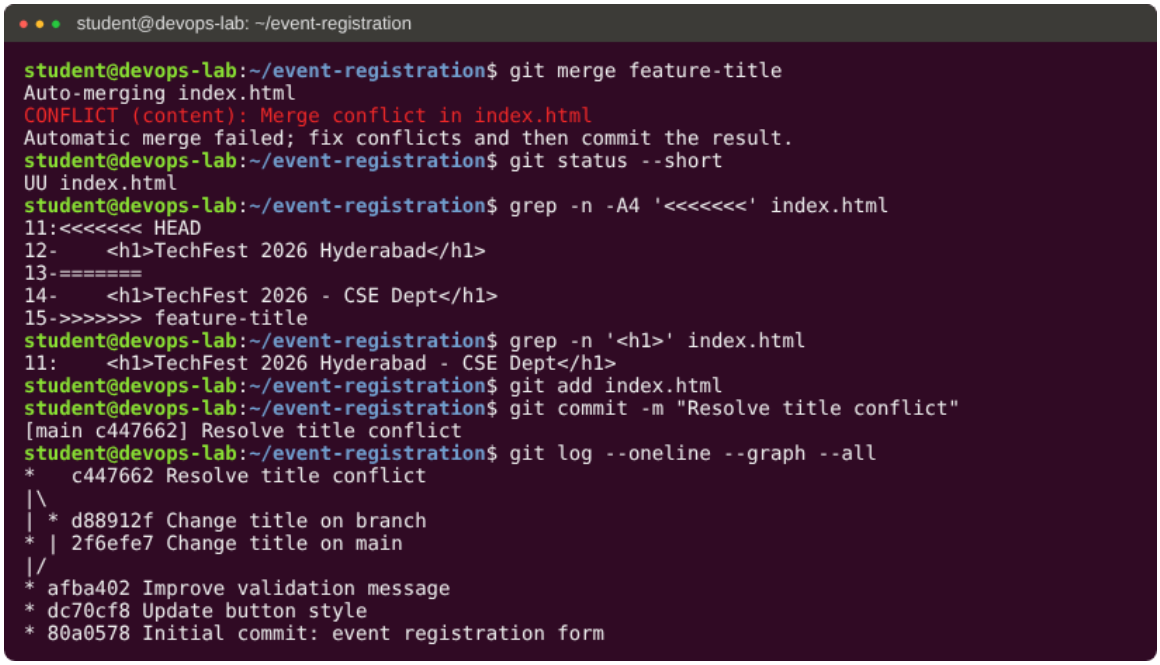
5. Create a feature branch, add phone validation in `script.js`, commit and push the branch:

```
git checkout -b feature-validation
git commit -am "Add phone number validation"
git push -u origin feature-validation
```

6. On GitHub, click **Compare & pull request**, add a title and description, click **Create pull request**, then **Merge pull request → Confirm merge**.
7. Update the local `main` with `git checkout main` and `git pull origin main`.
8. Create a merge conflict: on branch `feature-title` change the `<h1>` to "TechFest 2026 - CSE Dept"; on `main` change the same line to "TechFest 2026 Hyderabad"; commit both and run `git merge feature-title`.
9. Open `index.html`, keep the correct line, delete the conflict markers, then `git add index.html` and `git commit -m "Resolve title conflict"`, and push.
10. View history with `git log --oneline --graph --all` and on GitHub under **Commits** and **Network**.

Output:





Experiment 4: Jenkins Installation, Setup and Environment Exploration

Aim: To install and configure Jenkins and explore its dashboard, plugins, tools and node configuration.

Software required: Java 21 (OpenJDK), Jenkins LTS, a web browser; Windows 10/11 or Ubuntu 22.04.

Theory: Jenkins is an open-source automation server written in Java. It automates building, testing and deploying software, and is the most widely used tool for Continuous Integration (CI). Work is organised as jobs (Freestyle or Pipeline); each run of a job is a build. Jenkins is extended through plugins (Git, Pipeline, Docker, etc.) and runs by default on port 8080.

Procedure (Ubuntu):

1. Install Java and verify:

```
sudo apt update
sudo apt install fontconfig openjdk-21-jre -y
java -version
```

2. Add the Jenkins repository key and source, then install Jenkins:

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install jenkins -y
```

3. Start Jenkins and check its status:

```
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins
```

4. Read the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

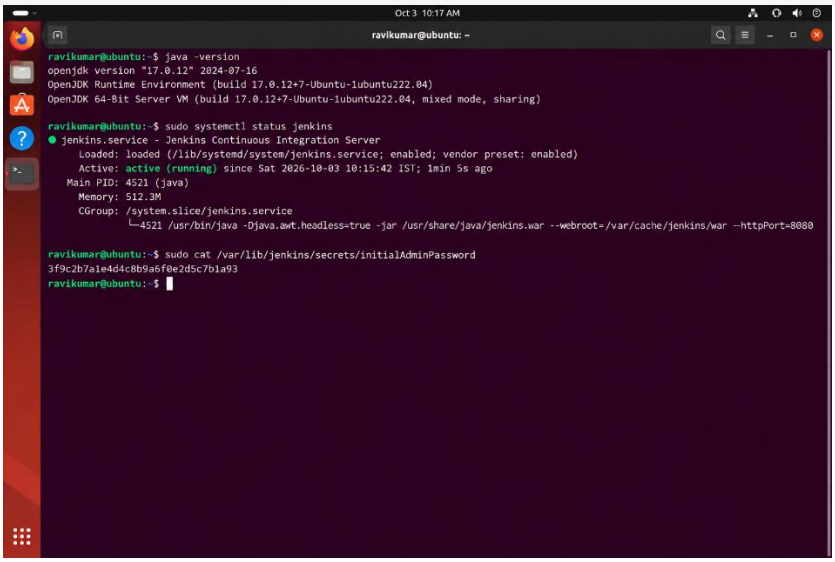
Procedure (Windows): Install JDK 17, download the Jenkins LTS `.msi` from jenkins.io/download, run it, choose "Run service as LocalSystem", keep port 8080, select the JDK path and finish. The password is in `C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword`.

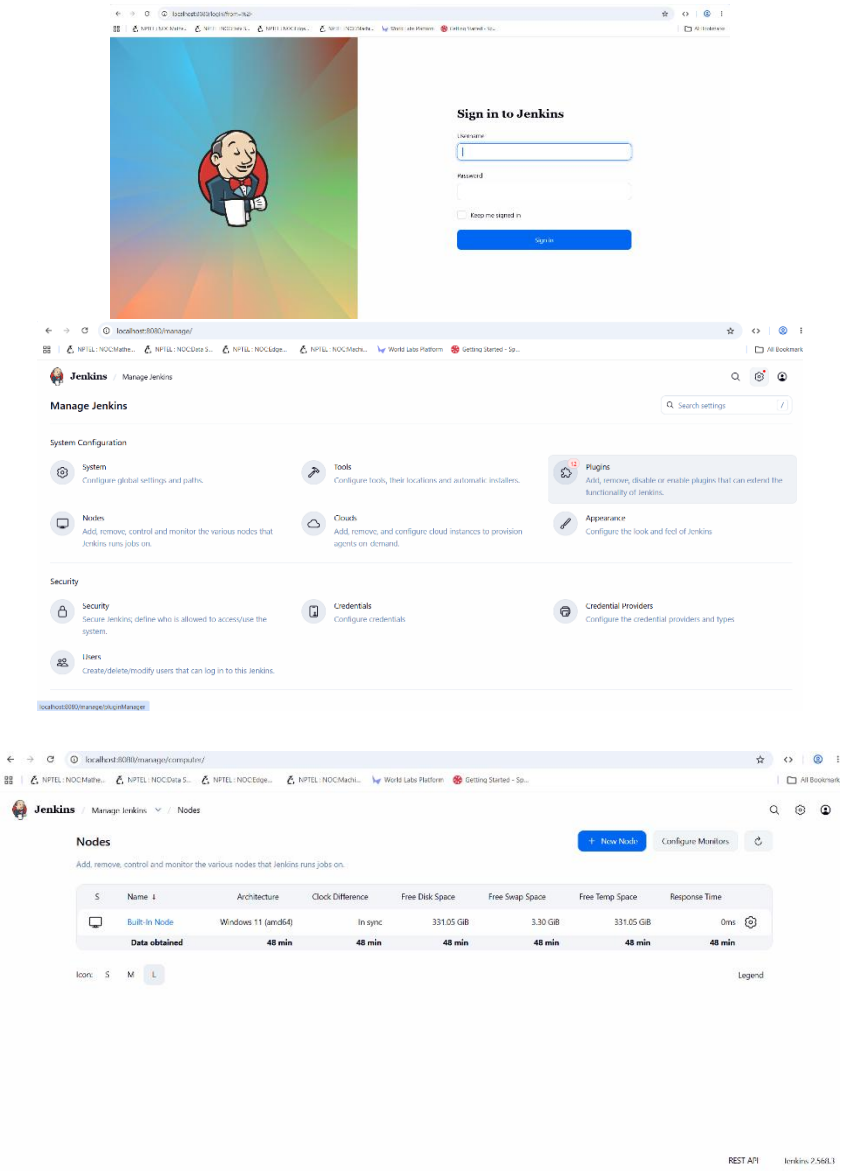
Setup (both platforms):

1. Open `http://localhost:8080` in a browser and paste the initial admin password.
2. Click **Install suggested plugins** and wait for installation to complete.
3. Create the first admin user (username, password, full name, email).
4. Keep the Jenkins URL as `http://localhost:8080/` and click **Start using Jenkins**.

Exploring the environment:

Menu (Dashboard → …)	What was observed
New Item	Job types: Freestyle project, Pipeline, Multibranch Pipeline, Folder, Organization Folder
Build History	Timeline of all builds with status
Manage Jenkins → System	Jenkins URL, number of executors, global environment variables, email settings
Manage Jenkins → Tools	JDK, Git, Maven, Gradle installations
Manage Jenkins → Plugins	Installed, Available and Updates tabs (Git, Pipeline, GitHub, Credentials installed)
Manage Jenkins → Nodes	Built-In Node with 2 executors, its free disk space and response time
Manage Jenkins → Credentials	Stored usernames/passwords, tokens and SSH keys
Manage Jenkins → Security / Users	Authentication settings and user accounts
My Views	Custom list views of jobs





Result: Jenkins was installed and configured successfully, and its dashboard, job types, plugins, tools, nodes and credentials were explored.

Experiment 5: Continuous Integration and Continuous Deployment using Jenkins

Aim: To demonstrate Continuous Integration and Continuous Deployment (CI/CD) using Jenkins by automatically building, testing and deploying the event registration form from GitHub whenever code is pushed.

Software required: Jenkins (from Experiment 4), Git, GitHub repository `event-registration` (from Experiment 3), Apache or Nginx web server (optional, for deployment).

Theory: Continuous Integration means every code change pushed to the shared repository is automatically built and tested, so errors are caught early. Continuous Delivery/Deployment extends this by automatically releasing the tested build to a server. In Jenkins, a **Freestyle job** is configured through the web form, while a **Pipeline job** is written as code in a `Jenkinsfile` with stages such as Checkout, Build, Test and Deploy. Builds are triggered by **Poll SCM** (Jenkins checks GitHub on a schedule) or by a **GitHub webhook** (GitHub notifies Jenkins on every push).

Part A: Freestyle job (CI)

Procedure:

1. Dashboard → **New Item** → name `EventReg-Freestyle` → **Freestyle project** → OK.
2. **Source Code Management** → Git → Repository URL `https://github.com/<username>/event-registration.git`, branch `*/main`.
3. **Build Triggers** → tick **Poll SCM** → Schedule `H/2 * * * *` (check every 2 minutes).
4. **Build Steps** → Add build step → **Execute shell** (Windows: **Execute Windows batch command**) and enter:

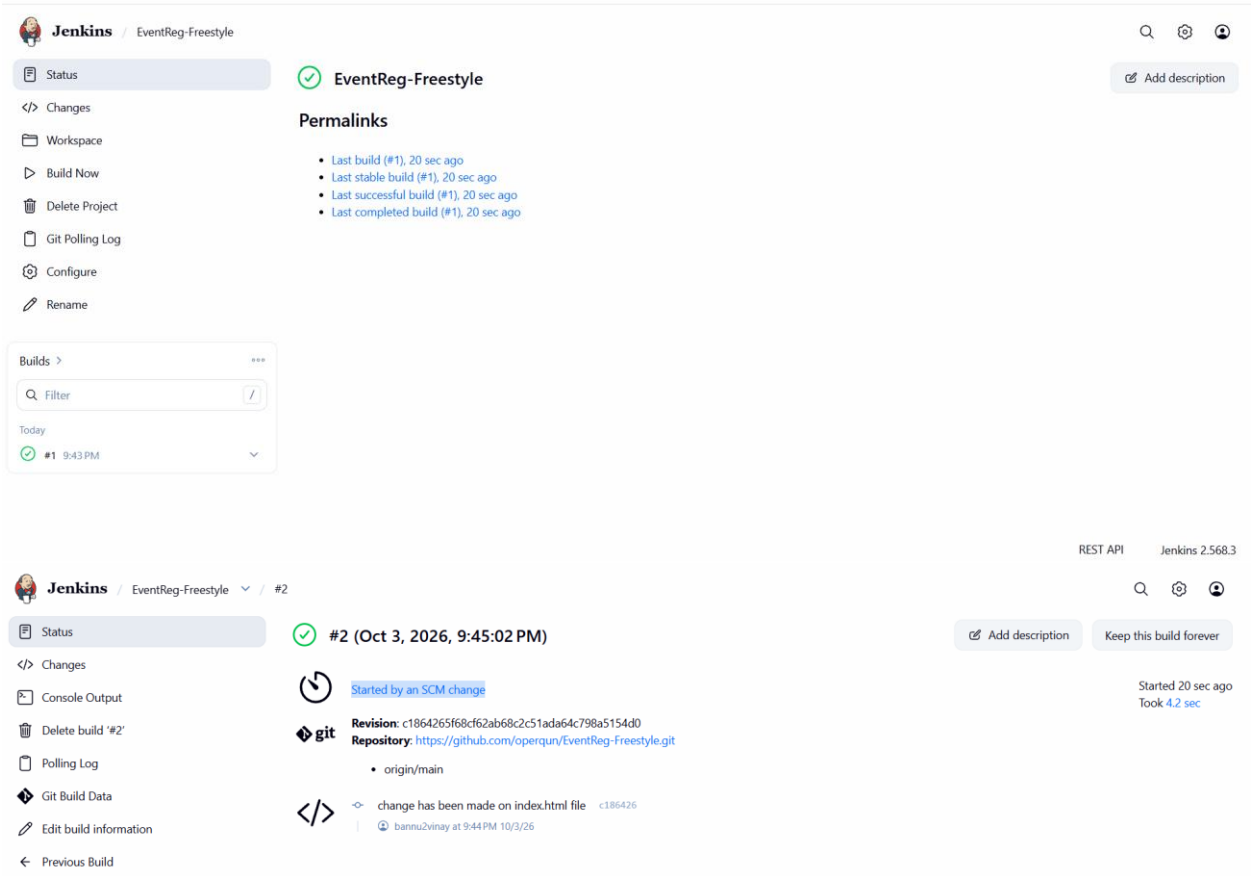
```
echo "Build started for Event Registration Form"
ls -l
test -f index.html && echo "index.html found"
test -f style.css  && echo "style.css found"
test -f script.js  && echo "script.js found"
echo "Build successful"
```

5. Click **Save**, then **Build Now**. Open the build number → **Console Output**.
6. Edit any file locally, `git commit` and `git push`. Within 2 minutes a new build starts automatically.

Reference console output (replace with your own screenshot, Fig 5.3):

```
Started by user Ravi Kumar
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/EventReg-Freestyle
The recommended git tool is: NONE
 > git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/EventReg-
Freestyle/.git # timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/ravikumar/event-
registration.git # timeout=10
Fetching upstream changes from https://github.com/ravikumar/event-registration.git
 > git fetch --tags --force --progress -- https://github.com/ravikumar/event-
registration.git +refs/heads/*:refs/remotes/origin/* # timeout=10
Checking out Revision 88a9bc25a85088cc40a9a46b279bea72240323be
(refs/remotes/origin/main)
Commit message: "Add phone number validation"
[EventReg-Freestyle] $ /bin/sh -xe /tmp/jenkins1234567890.sh
+ echo Build started for Event Registration Form
Build started for Event Registration Form
+ ls -l
total 12
-rw-r--r-- 1 jenkins jenkins 2145 Oct  3 11:02 index.html
-rw-r--r-- 1 jenkins jenkins 1702 Oct  3 11:02 script.js
-rw-r--r-- 1 jenkins jenkins 1110 Oct  3 11:02 style.css
+ test -f index.html
+ echo index.html found
index.html found
+ test -f style.css
+ echo style.css found
style.css found
+ test -f script.js
+ echo script.js found
script.js found
+ echo Build successful
Build successful
Finished: SUCCESS
```

After the push in step 6, the next build shows `Started by an SCM change` on its first line.



Part B: Pipeline job (CI/CD)

Procedure:

1. In the project folder create a file named `Jenkinsfile` with the code below, then commit and push it to GitHub.
2. Dashboard → **New Item** → name **`CI/CD pipeline`** → **Pipeline** → OK.
3. **Build Triggers** → tick **Poll SCM** → `H/2 * * * *` (or **GitHub hook trigger for GITScm polling** if a webhook is configured).
4. **Pipeline** → Definition **Pipeline script from SCM** → SCM **Git** → repository URL → branch `*/main` → Script Path `Jenkinsfile` → **Save**.
5. Click **Build Now** and observe the **Stage View** and **Console Output**.

Jenkinsfile:

```
pipeline {
    agent any

    environment {
        DEPLOY_DIR = '/var/www/html/event-registration'
    }
```

```
    stages {
        stage('Checkout') {
            steps {
                echo 'Fetching code from GitHub...'
                checkout scm
            }
        }
        stage('Build') {
            steps {
                echo 'Building the Event Registration Form...'
                sh 'ls -l'
            }
        }
        stage('Test') {
            steps {
                echo 'Running basic tests...'
                sh 'test -f index.html && test -f style.css && test -f script.js'
                sh 'grep -q "<form" index.html && echo "Form tag present"'
                sh 'grep -q "addEventListener" script.js && echo "Validation
script present"'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying to web server...'
                sh 'mkdir -p $DEPLOY_DIR'
                sh 'cp index.html style.css script.js $DEPLOY_DIR/'
                echo 'Deployed at http://localhost/event-registration/'
            }
        }
    }

    post {
        success { echo 'Pipeline completed successfully!' }
        failure { echo 'Pipeline failed. Check the logs.' }
    }
}
```

// leave space for output

Result: Continuous Integration and Continuous Deployment were demonstrated using Jenkins: every push to GitHub automatically triggered a build, tests verified the code, failures were reported immediately, and successful builds were deployed to the web server.

Experiment 6: Docker Commands for Content Management

Aim: To explore Docker commands for managing images, containers, volumes and networks, and to package and serve the event registration form as web content in a Docker container.

Software required: Docker Engine (Ubuntu) or Docker Desktop (Windows/macOS), Docker Hub account, source code of Experiment 1.

Theory: Docker is a containerisation platform that packages an application with everything it needs into a lightweight, portable **container**. An **image** is a read-only template; a container is a running instance of an image. A **Dockerfile** lists the steps to build an image. **Volumes** store content outside the container so it survives restarts, and **Docker Hub** is a public registry for sharing images. Compared with virtual machines, containers share the host OS kernel, so they start in seconds and use less memory.

Installation (Ubuntu):

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl enable --now docker
sudo usermod -aG docker $USER   # log out and log in again
```

Commands explored:

Category	Command	Purpose
System	`docker --version` / `docker info`	Show version / system-wide details
Images	`docker pull nginx`	Download an image from Docker Hub
Images	`docker images`	List local images
Images	`docker build -t <name> .`	Build an image from a Dockerfile
Images	`docker tag` / `docker push`	Tag / upload an image to Docker Hub
Images	`docker rmi <image>`	Remove an image
Containers	`docker run -d -p 8081:80 --name web nginx`	Create and start a container in background with port mapping
Containers	`docker ps` / `docker ps -a`	List running / all containers
Containers	`docker stop` / `docker start` / `docker restart`	Control container state
Containers	`docker exec -it <ctr> sh`	Open a shell inside a running container
Containers	`docker logs <ctr>`	View container output
Containers	`docker cp <src> <ctr>:<dest>`	Copy files into or out of a container

Category	Command	Purpose
Containers	`docker inspect <ctr>`	Show detailed JSON configuration
Containers	`docker rm <ctr>`	Remove a stopped container
Volumes	`docker volume create` / `ls` / `inspect`	Manage persistent content storage
Networks	`docker network ls` / `create`	Manage container networks
Cleanup	`docker system prune`	Remove unused containers, networks and images

Procedure:

1. Verify the installation with `docker --version` and run `docker run hello-world`.
2. Pull the Nginx web server image and list images.
3. Run an node container on port 3000 and open `http://localhost:3000` (default nodejs page).
4. Copy the server.js, package.js files into the running container with `docker cp` and refresh the browser.
5. Use `exec`, `logs`, `inspect`, `stop`, `start` and `rm` on the container.
6. In the `project` folder, create the `Dockerfile` below, build a custom image and run it.
7. Tag and push the image to Docker Hub.
8. Clean up unused objects.

Dockerfile:

```
FROM node:20

##Start with a Node.js 20 image.


WORKDIR /app

# the working directory inside the image/container.


COPY package.json .


# Copy package.json from the build context into the current working directory
inside the image.

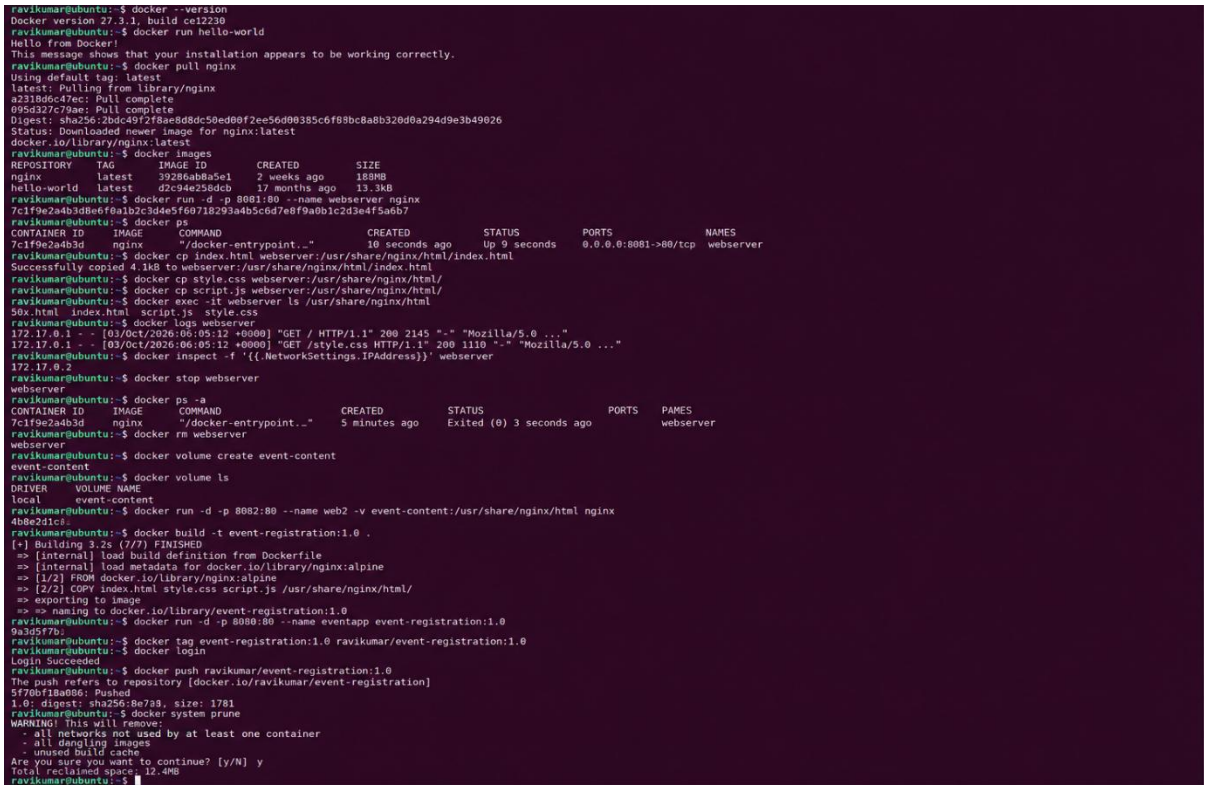

COPY server.js .


EXPOSE 3000

#describes the application's container port.
```

```
CMD ["node", "server.js"]
```

Output:



In Docker Desktop (Windows), the **Containers** and **Images** tabs are also good screenshots to include.

Result: Docker commands for managing images, containers, volumes and networks were explored, and the event registration form was packaged as a Docker image, served from a container, and pushed to Docker Hub successfully.

Experiment 7: Simple Containerized Application using Docker

Aim: To develop a simple Node.js event registration web application and containerize it using Docker.

Software required: Node.js 20+, npm, Docker Engine / Docker Desktop, VS Code, web browser.

Theory: A containerized application is packaged with its runtime, libraries and configuration into a Docker image, so it runs identically on any machine with Docker. The app here is built with **Express**, a lightweight Node.js web framework. It serves the registration form from Experiment 1 and exposes a small REST API: `POST /api/register` adds a participant, `GET /api/registrations` lists them, and `GET /health` reports the app status and the host (container) name. The `Dockerfile` builds the image in layers, and `HEALTHCHECK` lets Docker monitor the container.

Project structure:

```
event-app/
├── Dockerfile
├── .dockerignore
```

```
├── package.json
├── server.js
└── public/
    ├── index.html
    ├── style.css
    └── script.js
```

Program:

package.json

```
{
  "name": "event-registration-app",
  "version": "1.0.0",
  "description": "TechFest 2026 event registration app (containerized with
Docker)",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.21.0"
  }
}
```

Server.js

```
const http = require("http"); // loads Node.js's built-in HTTP module.

const server = http.createServer((req, res) => { //

    res.writeHead(200, {            //200:The request was successful.
        "Content-Type": "text/html"   // tells the browser: The response contains
HTML

    });
            // sends the HTML back to the browser.
    res.end(`
        <html>
            <head>
                <title>Docker Application</title>
            </head>

            <body>
                <h1>Hello from Docker!</h1>
                <p>
                    This application is running
                    inside a container.
                </p>
            </body>
        </html>
    `);
});

server.listen(3000, () => {
    console.log(
        "Server running on port 3000"
    );
});
```

`Dockerfile`

```
# Base image with Node.js 20 on Alpine Linux
FROM node:20-alpine

# Working directory inside the container
WORKDIR /app

# Install dependencies first (better layer caching)
COPY package.json .


# Copy application source
COPY server.js .

# Application port
EXPOSE 3000

# Container health check
HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget -qO- http://localhost:3000/health || exit 1

# Start the app
CMD ["node", "server.js"]
```

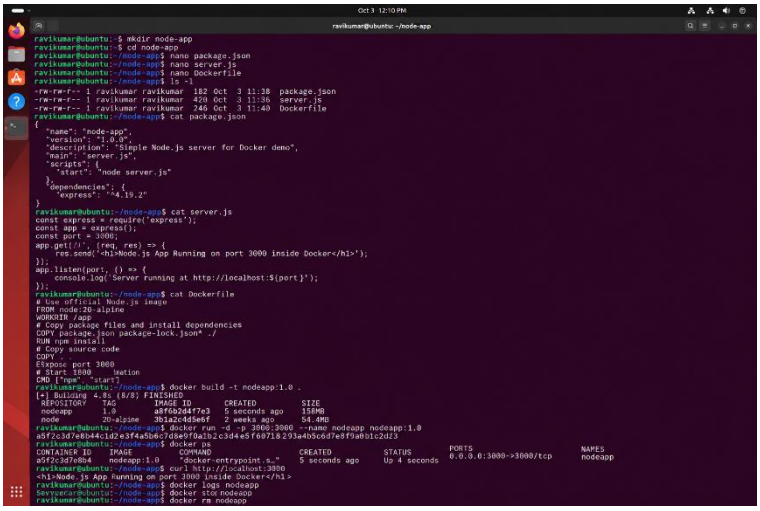
`.dockerignore`

```
node_modules
npm-debug.log
.git
```

Procedure:

1. Create the folder structure above and type in all the files.
2. Test the app without Docker: `node server.js`, and open `http://localhost:3000`.
3. Build the Docker image: `docker build -t event-app:1.0 .`
4. List images: `docker images`
5. Run the container: `docker run -d -p 3000:3000 --name node-server node-server:1.0`
6. Check it: `docker ps`, `docker logs node-server`, and open `http://localhost:3000`.
7. Register two participants, then try the same email again.
8. Stop and remove: `docker stop event-app` and `docker rm event-app`.

Output:



Result: A simple Node.js event registration application was developed, tested, and containerized using a Dockerfile; the container was built, run with port mapping and health checking, and pushed to Docker Hub.

Experiment 8: Integrating Kubernetes and Docker

Aim: To install Kubernetes (Minikube and kubectl), integrate it with Docker, and run Docker images as pods in a Kubernetes cluster.

Software required: Docker Engine / Docker Desktop, Minikube, kubectl, at least 2 CPUs, 4 GB RAM and 20 GB free disk.

Theory: Docker builds and runs individual containers; Kubernetes (K8s) **orchestrates** many containers across machines – it schedules them, restarts failed ones, scales them and load-balances traffic. Docker produces the image; Kubernetes runs it. **Minikube** creates a single-node Kubernetes cluster on a laptop, and with `--driver=docker` the whole cluster runs inside a Docker container. **kubectl** is the command-line client that talks to the cluster's API server.

Key Kubernetes objects:

Object	Purpose
Pod	Smallest unit; one or more containers sharing network and storage
Deployment	Keeps the desired number of pod replicas running; handles rolling updates
ReplicaSet	Created by a Deployment to maintain the replica count
Service	Stable network address and load balancer for a set of pods (ClusterIP, NodePort, LoadBalancer)
Namespace	Logical partition of cluster resources

Object	Purpose
Node	A machine (VM or container) that runs pods

Procedure:

1. Install kubectl (Ubuntu):

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
```

2. Install Minikube (Ubuntu):

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-
amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version
```

On Windows, install Docker Desktop, then run `winget install Kubernetes.minikube` and `winget install Kubernetes.kubectl` (or enable **Settings → Kubernetes → Enable Kubernetes** in Docker Desktop).

3. Start the cluster using Docker as the driver, and check it:

```
minikube start --driver=docker
minikube status
kubectl cluster-info
kubectl get nodes
docker ps            # the Minikube node itself runs as a Docker container
```

4. Run a Docker Hub image as a pod and expose it:

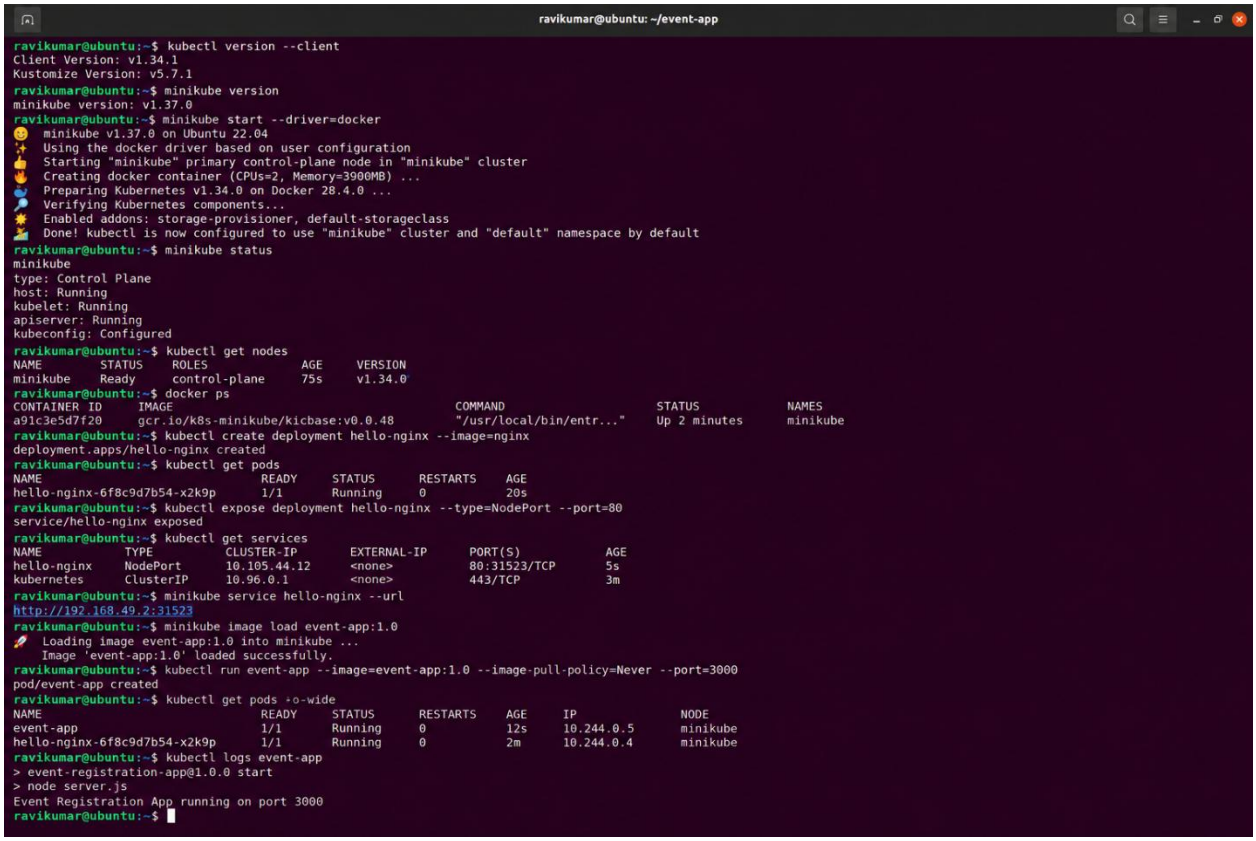
```
kubectl create deployment hello-nginx --image=nginx
kubectl get pods
kubectl expose deployment hello-nginx --type=NodePort --port=80
kubectl get services
minikube service hello-nginx --url
```

5. Use a locally built Docker image (from Experiment 7) inside Kubernetes:

```
minikube image load event-app:1.0
minikube image ls | grep event-app
kubectl run event-app --image=event-app:1.0 --image-pull-policy=Never --port=3000
kubectl get pods -o wide
kubectl logs event-app
kubectl port-forward pod/event-app 3000:3000    # open http://localhost:3000
```

6. Explore the cluster: `kubectl describe pod event-app`, `kubectl get all`, `kubectl get namespaces`, `minikube dashboard`.
7. Clean up: `kubectl delete pod event-app`, `kubectl delete deployment hello-nginx`, `kubectl delete service hello-nginx`.

Output:



Result: Kubernetes was installed using Minikube with the Docker driver and integrated with Docker; public and locally built Docker images were run as pods, exposed as services and accessed from the browser.

Experiment 9: Automating the Containerized Application using Kubernetes

Aim: To automate the deployment, scaling, self-healing and updating of the containerized event registration application (Experiment 7) using Kubernetes Deployment and Service manifests.

Software required: Minikube and kubectl (Experiment 8), Docker, the `event-app` image from Experiment 7.

Theory: Kubernetes works **declaratively**: you describe the desired state in YAML files and Kubernetes continuously makes the cluster match it. A **Deployment** with `replicas: 3` keeps three pods running – if a pod crashes or is deleted, a new one is created automatically (self-healing). **Liveness** and **readiness probes** call the app's `/health` endpoint so Kubernetes restarts unhealthy containers and only sends traffic to ready ones. A **Service** of type NodePort load-balances requests across the pods. Changing the image triggers a **rolling update** with zero downtime, and `kubectl rollout undo` rolls back. A shell script ties the steps together so the whole process runs with one command.

Program:

`k8s/deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: event-app
  labels:
```

```
    app: event-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: event-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      labels:
        app: event-app
    spec:
      containers:
        - name: event-app
          image: event-app:1.0          # or <dockerhub-username>/event-app:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 3000
          env:
            - name: PORT
              value: "3000"
          resources:
            requests:
              cpu: "100m"
              memory: "64Mi"
            limits:
              cpu: "250m"
              memory: "128Mi"
          readinessProbe:
            httpGet:
              path: /health
              port: 3000
            initialDelaySeconds: 3
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: 3000
            initialDelaySeconds: 10
            periodSeconds: 10
```

`k8s/service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: event-app-service
spec:
  type: NodePort
  selector:
    app: event-app
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30080
```

`deploy.sh` (automation script)

```
#!/bin/bash
# Automates build -> load -> deploy -> verify for the Event Registration App
set -e
IMAGE="event-app:${1:-1.0}"

echo "Step 1: Building Docker image $IMAGE"
docker build -t $IMAGE .

echo "Step 2: Loading image into Minikube"
minikube image load $IMAGE

echo "Step 3: Applying Kubernetes manifests"
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl set image deployment/event-app event-app=$IMAGE

echo "Step 4: Waiting for rollout to finish"
kubectl rollout status deployment/event-app --timeout=120s

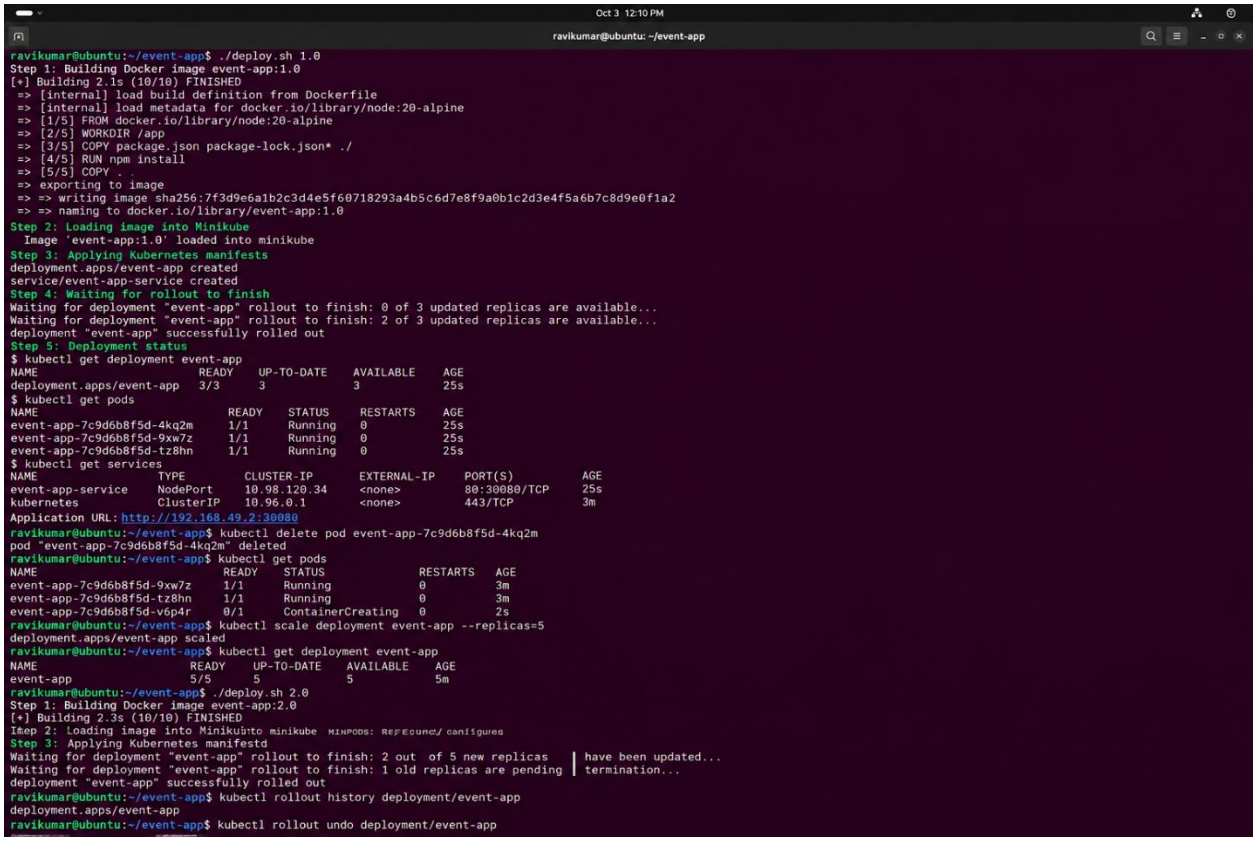
echo "Step 5: Deployment status"
kubectl get deployment,pods,service -l app=event-app
kubectl get service event-app-service

echo "Application URL: $(minikube service event-app-service --url)"
```

Procedure:

1. Inside the `event-app` folder from Experiment 7, create a `k8s` folder with `deployment.yaml` and `service.yaml`, and create `deploy.sh` in the project root.
2. Make the script executable and run it: `chmod +x deploy.sh` then `./deploy.sh 1.0`.
3. Open the printed URL (or run `minikube service event-app-service`). Refresh several times – the "Served by" pod name changes as the Service load-balances across pods.
4. **Self-healing:** delete a pod with `kubectl delete pod <pod-name>` and immediately run `kubectl get pods -w`; a replacement pod is created.
5. **Scaling:** `kubectl scale deployment event-app --replicas=5`, then `kubectl get pods`.
6. **Rolling update:** change the `<h1>` in `public/index.html` to "TechFest 2026 – Registrations Open", then run `./deploy.sh 2.0` and watch `kubectl rollout status`.
7. **History and rollback:** `kubectl rollout history deployment/event-app`, then `kubectl rollout undo deployment/event-app`.
8. Optional **autoscaling:** `minikube addons enable metrics-server`, then `kubectl autoscale deployment event-app --cpu-percent=50 --min=2 --max=6` and `kubectl get hpa`.
9. Clean up: `kubectl delete -f k8s/`.

Output:



Result: Running the containerized event registration application was automated with Kubernetes: a single script built and deployed it, and Kubernetes kept 3 replicas running, replaced deleted pods automatically, scaled on demand, load-balanced traffic and performed a zero-downtime rolling update and rollback.